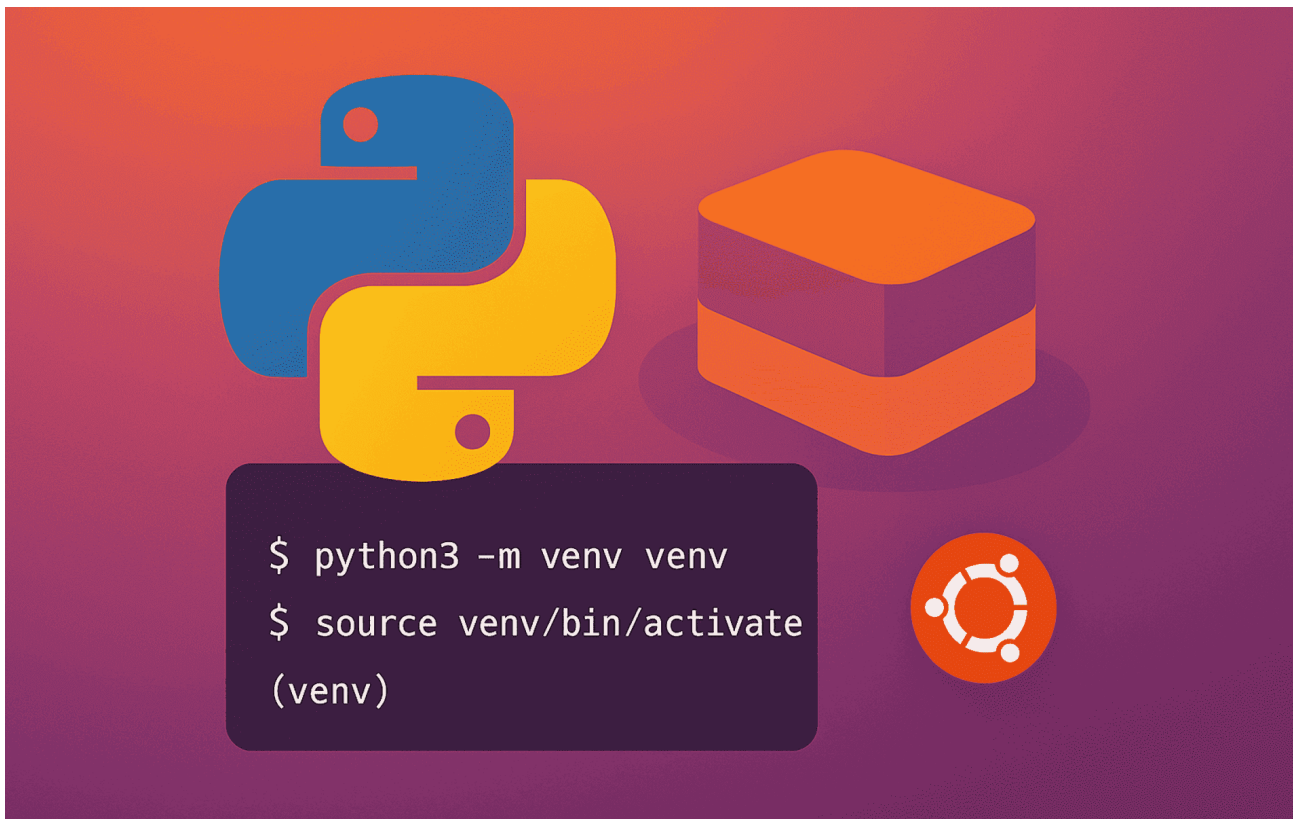


Create a Python Virtual Environment on Ubuntu

 linuxcapable.com/how-to-create-a-python-virtual-environment-on-ubuntu-linux

Joshua James

July 15, 2024



Ubuntu protects its system Python more aggressively now, so the clean way to install project libraries is to keep them out of `/usr/lib/python3/dist-packages`. When you create a Python virtual environment on Ubuntu, each project gets its own interpreter path, pip install location, and dependency set without touching the packages APT manages.

The built-in `venv` module is the default choice for most Ubuntu projects. It works on Ubuntu 26.04 LTS (Resolute Raccoon), 24.04 LTS (Noble Numbat), and 22.04 LTS (Jammy Jellyfish), while the optional `virtualenv` package remains useful when you need its extra seeding or interpreter-selection features.

A complete project workflow starts with `python3-venv`, a project-local `.venv` directory, activation, safe pip installs, and a reproducible `requirements.txt` file. Ubuntu 24.04 and newer releases can also show the common `externally-managed-environment` error when pip runs outside that isolated environment.

Ubuntu 26.04 currently uses Python 3.14.x as the default `python3` interpreter, Ubuntu 24.04 uses Python 3.12.x, and Ubuntu 22.04 uses Python 3.10.x. The commands are the same across these releases, but version output follows the interpreter on your system.

Update Ubuntu Package Lists

Refresh your local APT metadata before installing the Python virtual environment package:

```
sudo apt update
```

These commands use `sudo` for package installation. If your account is not allowed to use `sudo` yet, run the commands as root or follow the guide on [adding a user to sudoers on Ubuntu](#).

Install python3-venv on Ubuntu

Most Ubuntu desktop and server installs already include `python3`, but the `venv` module usually needs the separate `python3-venv` package. Install both package names so minimal systems are covered too:

```
sudo apt install python3 python3-venv
```

The `python3-venv` package comes from Ubuntu's Universe component on current LTS releases. If APT cannot find the package on a customized system, enable Universe first with the [Ubuntu Universe and Multiverse guide](#), then repeat the install command.

Check Python and venv Versions

Confirm the default interpreter before creating the environment:

```
python3 --version
```

Relevant output by Ubuntu release:

Ubuntu Release	Default Python Branch	venv Package Branch
Ubuntu 26.04 LTS (Resolute Raccoon)	Python 3.14.x	<code>python3.14-venv</code>
Ubuntu 24.04 LTS (Noble Numbat)	Python 3.12.x	<code>python3.12-venv</code>
Ubuntu 22.04 LTS (Jammy Jellyfish)	Python 3.10.x	<code>python3.10-venv</code>

The unversioned `python3-venv` package follows Ubuntu's default `python3` branch. It does not create an older or newer interpreter by itself.

Create the .venv Directory

Create a project directory, move into it, then create a virtual environment named `.venv`. The `mkdir -p` command creates the parent path when it does not already exist; the [mkdir](#)

[command examples](#) cover the flag in more detail.

```
mkdir -p ~/projects/demo-venv
cd ~/projects/demo-venv
python3 -m venv .venv
```

The command normally finishes without printing output. That quiet result is expected.

Activate the Python Virtual Environment

Activate the environment in your current shell session:

```
source .venv/bin/activate
```

Your prompt usually gains a (.venv) prefix. Confirm the active environment, interpreter path, and pip location:

```
echo "$VIRTUAL_ENV"
command -v python3
python -m pip --version
```

Relevant output pattern:

```
/home/username/projects/demo-venv/.venv
/home/username/projects/demo-venv/.venv/bin/python3
pip [version] from /home/username/projects/demo-venv/.venv/lib/python3.x/site-
packages/pip (python 3.x)
```

The important detail is the path. If the first two lines point inside .venv, pip installs packages into the virtual environment instead of Ubuntu's system Python directory.

Deactivate the Virtual Environment

Leave the environment when you are finished working in the project:

```
deactivate
```

Closing the terminal also ends the activated session. The .venv directory remains on disk until you delete it.

Install Python Packages Inside venv

Upgrade pip Tooling When Needed

After activation, python and pip refer to the virtual environment. Upgrade pip, setuptools, and wheel inside the environment when you need newer packaging behavior:

```
python -m pip install --upgrade pip setuptools wheel
```

The exact package versions change over time, so do not treat a specific pip version as required unless your project says so. A successful run ends with a Successfully installed line naming the upgraded packages.

Install a Package with pip

Install packages only while the environment is active. Replace <package_name> with the library your project needs:

```
python -m pip install <package_name>
```

For a quick test package, install requests:

```
python -m pip install requests
```

Verify the package imports from the active environment:

```
python -c "import requests; print(requests.__version__)"
```

This prints the installed requests version. The number can change as PyPI releases new versions.

Do not use sudo pip. On Ubuntu 24.04 and 26.04, pip outside a virtual environment is blocked by [PEP 668](#) externally managed environment protection. On Ubuntu 22.04, sudo pip may still run, but it can overwrite files that APT expects to control.

Save Dependencies to requirements.txt

Record the active environment's packages when the project is ready to share or deploy:

```
python -m pip freeze > requirements.txt
```

The file contains exact package pins. A small example might look like this:

```
requests==2.33.1
```

Those version numbers are examples. Your file should reflect the packages installed in your own environment, including any transitive dependencies pip records.

Install the same dependency set in a fresh environment with:

```
python -m pip install -r requirements.txt
```

For a broader pip workflow, including package search, upgrades, and cleanup, use the [Python pip guide for Ubuntu](#).

Uninstall a Package from venv

Remove packages from the active virtual environment with pip:

```
python -m pip uninstall requests
```

pip prompts before deleting files. Add `-y` only when you intentionally want a non-interactive uninstall:

```
python -m pip uninstall -y requests
```

Use virtualenv on Ubuntu When Needed

The built-in `venv` module is enough for most current Ubuntu projects. Use `virtualenv` when you specifically need its faster environment seeding, broader activation script support, or advanced interpreter selection behavior.

Install virtualenv on Ubuntu

Install Ubuntu's packaged `virtualenv` tool with APT:

```
sudo apt install python3-virtualenv
```

Check the installed command:

```
virtualenv --version
```

Ubuntu package versions differ by release, so expect a `virtualenv 20.x` line rather than one fixed patch number.

Create an Environment with virtualenv

Create and activate a `virtualenv` environment the same way you would a `venv` environment:

```
virtualenv .venv
source .venv/bin/activate
```

To target a specific installed interpreter, pass the interpreter name with `-p`:

```
virtualenv -p python3.12 .venv
```

The named interpreter must already exist on the system. For example, Ubuntu 24.04 does not create a Python 3.10 environment from `python3-venv` alone; install Python 3.10 first, then create the environment with `python3.10 -m venv .venv`. Use the [Python 3.10 on Ubuntu guide](#) for that interpreter setup.

Compare python3-venv, python3-dev, and python3-devel

Search results often mix Ubuntu and Fedora package names. Fedora's `python3-devel` package is closest to Ubuntu's `python3-dev`, but neither package is required just to create a virtual environment with `python3 -m venv`.

On Ubuntu 26.04 and 24.04, `python3-venv` depends on the matching versioned `venv` package and `python3`. It does not depend on `python3-dev`. Development headers become relevant when a pip package needs to compile native extensions, and APT may also recommend them when installing broader tooling such as `python3-pip` or `python3-virtualenv`.

Common Python venv Workflows

New Project Setup

For a new application, create the project directory, activate the environment, install dependencies, then save the dependency file:

```
mkdir -p ~/projects/myproject
cd ~/projects/myproject
python3 -m venv .venv
source .venv/bin/activate
python -m pip install requests flask
python -m pip freeze > requirements.txt
```

The `.venv` name is a common convention because it stays hidden in normal directory listings and is recognized by many editors. A plain `venv` directory also works; choose one pattern and use it consistently.

Clone an Existing Project

When a repository already includes `requirements.txt`, clone it, create a fresh environment, and install the pinned dependencies. If Git is not installed yet, use the [Git on Ubuntu installation guide](#) first.

```
git clone https://github.com/user/project.git
cd project
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

The example GitHub URL is a placeholder. Use your project's real repository URL.

Work Across Multiple Projects

Deactivate the current environment before switching to another project so your prompt and pip target stay clear:

```
cd ~/projects/project-a
source .venv/bin/activate
python manage.py runserver
deactivate
```

```
cd ~/projects/project-b
source .venv/bin/activate
python app.py
```

Each project keeps its own packages. One project can use Django 4.2 while another uses Django 5.x without forcing a global package change.

Keep .venv Out of Git

Commit `requirements.txt`, not the virtual environment directory. Add common environment names to `.gitignore`:

```
.venv/
venv/
env/
ENV/
```

Virtual environments contain system-specific interpreter paths and installed wheels. Recreating them from `requirements.txt` is cleaner than moving them between machines.

Troubleshoot Python Virtual Environment Errors

Fix externally-managed-environment

The externally-managed-environment error means pip is running against Ubuntu's system Python instead of an isolated environment.

Relevant error lines include:

```
error: externally-managed-environment
This environment is externally managed.
If you wish to install a non-Debian-packaged Python package, create a virtual
environment using python3 -m venv path/to/venv.
```

Create and activate the environment, then run pip again inside it:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install requests
```

Ubuntu's error text may mention `python3-full`. For this workflow, `python3-venv` is the smaller package that enables virtual environment creation. Use `python3-full` only when you intentionally want Ubuntu's fuller Python package set.

Ubuntu's error text may also mention `pipx`. That path is for standalone Python command-line applications, not normal project libraries; project dependencies should stay inside the project's `.venv`.

Fix ensurepip Is Not Available

If `python3 -m venv` reports that `ensurepip` is unavailable, the matching `venv` package is missing:

The virtual environment was not created successfully because `ensurepip` is not available.

Install `python3-venv`, then recreate the environment:

```
sudo apt install python3-venv
test -d .venv && mv .venv ".venv.broken.$(date +%Y%m%d%H%M%S)"
python3 -m venv .venv
source .venv/bin/activate
```

The guarded `mv` line keeps an existing broken environment as a timestamped backup instead of deleting it. The [mv command guide](#) covers safe rename and move behavior in more detail.

Fix No Module Named pip in .venv

If an existing environment reports that `pip` is missing, recreate it after installing `python3-venv`:

```
.venv/bin/python3: No module named pip

sudo apt install python3-venv
test -d .venv && mv .venv ".venv.broken.$(date +%Y%m%d%H%M%S)"
python3 -m venv .venv
source .venv/bin/activate
python -m pip --version
```

If you are using a separately installed Python version such as `python3.10` on Ubuntu 24.04, install that interpreter's matching `venv` support from the same source before creating the environment.

Fix Permission Denied During pip Install

A path under `/usr/local/lib`, `/usr/lib`, or `dist-packages` usually means the virtual environment is not active:

ERROR: Could not install packages due to an OSError: [Errno 13] Permission denied

Activate the environment and verify the `pip` path before trying again:

```
source .venv/bin/activate
python -m pip --version
python -m pip install requests
```

Remove Python Virtual Environments on Ubuntu

Delete a Project Virtual Environment

Removing `python3-venv` or `python3-virtualenv` does not remove environments you already created. Delete the project environment directory directly when you no longer need it.

The following command permanently deletes the selected virtual environment directory and every package installed inside it. Check your current directory and verify the target before running `rm -rf`.

```
pwd
ls -ld .venv
```

Run the deletion only after the listing shows the environment directory you intended to remove:

```
rm -rf .venv
```

If your environment uses another name, replace `.venv` with that directory name.

Remove virtualenv Package

If you installed the optional `virtualenv` package and no longer need it, remove it with APT:

```
sudo apt remove python3-virtualenv
```

Verify that no installed package row remains. The [grep command](#) filters for the `ii` status that marks installed Debian-family packages:

```
dpkg -l python3-virtualenv | grep '^ii' || echo "python3-virtualenv is not installed"
```

Preview dependency cleanup before running autoremove:

```
sudo apt autoremove --dry-run
```

Continue only if the preview lists packages you are comfortable removing:

```
sudo apt autoremove
```

Remove python3-venv Package

Keep `python3-venv` installed if you still create Python project environments. If you are cleaning a minimal system and know you no longer need `venv` support, remove it with:

```
sudo apt remove python3-venv
```

Verify the installed-state after removal:

```
dpkg -l python3-venv | grep '^ii' || echo "python3-venv is not installed"
```

Use the same autoremove preview pattern before deleting leftover dependencies.

Conclusion

Your Ubuntu projects can now install Python packages inside isolated `.venv` directories while APT keeps control of the system interpreter. Keep `requirements.txt` in Git, ignore the environment directory, and use the guide to [install Visual Studio Code on Ubuntu](#) or [install PyCharm on Ubuntu](#) when editor integration matters.